Міністерство освіти і науки України
Національний технічний університет України “КПІ” імені Ігоря Сікорського
Кафедра обчислювальної техніки ФІОТ

**ЗВІТ**
з Лабораторної роботи №3
з навчальної дисципліни “Технології Computer Vision (сертифікаційна програма Data Science із Sigma Software)”

Тема: **ФІЛЬТРАЦІЯ ТА ПОКРАЩЕННЯ ЯКОСТІ ЦИФРОВИХ ЗОБРАЖЕНЬ**

**Виконав:**
Студент 3-го курсу кафедри ОТ ФІОТ
Навчальної групи ІО-43
Шепітко Владислав
Варіант №17

**Перевірив:**
Професор кафедри ОТ ФІОТ
Олексій Писарчук

Київ-2026

**Мета**: Дослідити принципи та особливості практичного застосування технологій фільтрації та покращення якості цифрових зображень для задач Computer Vision з використанням спеціалізованих програмних бібліотек.
**Завдання**: За основу взято III рівень складності. Група вимог для завдання:

1) Доповнити результати Лабораторної роботи №2 методами фільтрації та корекції кольору цифрових зображень та розробити / модифікувати скрипт з функціоналом.
2) За даними від Esri | Landsat Explorer та Bing Maps підрахувати кількість будівель в головному кампусі КПІ.
3) Порівняти отримані результати. Знайти комбінацію операцій обробки растрових зображень та векторизації коли кількість знайдених будівель за Esri | Landsat Explorer буде відповідати реальній кількості будівель в головному кампусі КПІ.

*Еталон кількості будівель може бути як результат обробки зображень від* Bing Maps.

**Теоретичні відомості:**
Дана лабораторна робота включає в себе роботу з растровими зображеннями та векторизацію як інструмент для детекції границь об'єктів (за завданням це мають бути будівлі)

Для початку, розберемо різницю між джерелами супутникового знімку.

- **Bing Maps** - інтерактивна карта, що використовує супутникові знімки та аерофотографії для якісного зображення карти. Bing Maps дозволяє при максимально детальному масштабі отримати картинку, де ~29,8 см / піксель.
- **Esri | Landsat Explorer** - інтерактивна карта, що використовує знімки з супутників програми NASA Landsat. Ця карта носить науковий характер використання. При максимально детальному масштабі, можна отримати картинку, де 1 піксель дорівнює області 30x30 метрів.

Саме ці нюанси важливі для виконання завдання III рівня складності, так як стандартні методи детекції границь на карті **Esri | Landsat Explorer** - не працюють. Через це, як альтернативне джерело порівняння зображень було обрано **Google Maps**.

Розберемо одразу весь теоретичний матеріал, використаний для виконання лабораторної роботи №3.

1) **Моделі кольору**. В ході виконання роботи, ми ознайомились с моделями:

- **RGB** - стандартна колірна модель, яка містить три канали, для кодування червоного (R), зеленого (G), синього (B) кольорів. Саме ці кольори дають можливість відтворити весь спектр видимого світла. Кожен канал кодується 1 байтом (тобто 256 біт), що дозволяє отримати загалом 16 777 216 варіантів кольорів.

- **BGR** - колірна модель, яка історично була стандартом використання на Windows та деяких відеокартах. Широко використовується для обробки зображень бібліотекою OpenCV. З точку зору фізики процесу, вона нічим не відрізняється від моделі RGB. Основною їх відмінністю є порядок кольорових каналів.

- **HSV** - трьох-компонента колірна модель, яка описує колір звичним для людини чином. Складається з компонент відтінку (Hue), насиченості (Saturation) та значення яскравості (Value).

- **GRAYSCALE** - колірна модель, відтінків сірого. Найпопулярніше кодування для моделі відтінків сірого - 8-бітна модель. Вона дозволяє кодувати 256 градацій сірого кольору, від абсолютно чорного (0) і до абсолютно білого (255).

- **Монохром** - фактично являє собою GRAYSCALE, проте кодується всього 1 бітом (де 0 це чорний, а 1 - білий). Одна з найголовніших моделей представлення зображення у Computer Vision де факто.

2) **Алгоритми фільтрації**:
   - **Медіанний фільтр** - метод фільтрації, що використовується для очищення зображень від шумів. Математично, він є нелінійним фільтром. Алгоритм медіанного фільтру складається з наступних етапів:
     1) **Формування множин пікселів**. Нехай маємо матрицю пікселів з наступними значеннями:

$$W = \begin{pmatrix} 12 & 14 & 11 \\ 11 & 150 & 13 \\ 15 & 11 & 12 \end{pmatrix}$$

     2) **Сортування множини та знаходження медіани**. Бачимо, що серед пікселів приблизно одного діапазону, маємо шум у вигляді числа 150. Тому, відсортуємо масив значень за

зростанням. Тоді, наш шум опиниться у кінці списку, а значення для заміни нашого битого пікселю, буде 5-ий елемент у відсортованому списку (так як у нас матриця містить 9 елементів).

$$W_{sort} = (11, 11, 11, 12, 12, 13, 14, 15, 150)$$

Звідси бачимо, що замість значення 150, ми отримаємо значення 12.

- **Адаптивне вирівнювання гістограми (алгоритм CLAHE)** - метод обробки комп'ютерного зображення, що покращує локальну контрастність зображень, на якому є зони з різним рівнем освітленості, або якщо треба виділити приховані деталі у дуже світлих або темних зонах.

- **Фільтр Гаусса** - один з найпопулярніших фільтрів в Computer Vision. Фільтр Гаусса дозволяє згладжувати, знизити рівень деталізації зображення та прибирати високочастотний шум. В основі математичної реалізації фільтрації Гаусса, лежить нормальний розподіл величин. Давайте спробуємо вивести формулу фільтрації Гаусса:

  Розглянемо стандартну формулу нормального розподілу на одновимірній щільності вірогідності:

  $$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{x^2}{2\sigma^2}}$$

  Так як зображення, це двовимірний масив, то нехай розмиття фільтром для осі x та y буде виконуватись незалежно один від одного, та з однаковою силою ( $\sigma_x = \sigma_y = \sigma$ ).

  За законами теорії вірогідностей, щільність сумісного розподілу двух незалежних, випадкових величин дорівнює добутку їх окремих щільностей. Звідси:

  $$G(x, y) = (\frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{x^2}{2\sigma^2}}) \cdot (\frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{y^2}{2\sigma^2}})$$

  Перемножуємо щільності розподілу і отримаємо кінцеву формулу для Гауссового фильтра:

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

3) **Алгоритми бінаризації.** Для початку, бінаризація - це процес перетворення кольорового зображення (або зображення у відтінках сірого) у формат монохромного зображення, для легшої детекції границь.

   В ході виконання роботи ми використовували, або ознайомлювались з наступними методами:

   - **Метод Отцу** - один із найпопулярніших способів глобального переведення зображення у монохром. Сутність методу полягає в тому, що алгоритм аналізує гістограму зображення, для того, щоб ідеально підлаштувати поріг між відтінками і максимально эффективно розділити кольори на дві групи - темні та світлі.

   - **Адаптивна порогова фільтрація (бінаризація)** - метод цифрової обробки зображень, який перетворює кольорове зображення або картинку у градаціях сірого на бінарне. Головна особливість цього методу полягає в тому, що порогове значення яскравості не є однаковим для всього зображення, а обчислюється індивідуально для кожного пікселя на основі аналізу його сусідів (локальної області).

4) **Інші алгоритми комп'ютерного зору:**

   - **Метод детекції границь Кенні** - один із основних методів длля детекції границь об'єктів. Працює за принципом находження градієнтів змін кольорів. Там де відбувається різкий перепад яскравості і буде областю границі за алгоритмом Кенні.

   - **Морфологічні операції** - спеціальні операції, які призначені для зміни форми або структури об'єктів. В лабораторній роботі, я використовував операції:
     - **Ерозія** - операція, яка дозволяє стискати об'єкти.
     - **Дилітація** - операція, яка дозволяє розширювати об'єкти.
     - **Мофрологічна операція відкриття** - комбінована операція Ерозії та Дилітації, яка дозволяє видаляти шумні об'єкти, які не мають сенсу.
     - **Морфологічна операція закриття** - комбінована операція Дилітації та Ерозії, яка дозволяє з'єднувати сегментовані об'єкти в одне ціле (закривати дірки, з'єднувати об'єкти з невеликим розривом).

   - **Алгоритм Рамера - Дугласа - Пекера** - алгоритм, який зменшує кількість контрольних точок для кривої, видаляючи їх, якщо вони відхиляются від прямої менше, ніж за задане порогове значення.

- **Перетворення Хафа** - алгоритм, що дозволяє знаходити правильні примітивні фігури на кривій і спрощувати задану криву групою точок, що описують знайдену примітивну фігуру.

- **Сегментація методом вододілу** - алгоритм, який дозволяє розділяти об’єкти які дотикаються один до одного, або перетинаються. В лабораторній роботі, використання цього методу покращило результати детекції на 5-10%. Суть полягає в тому, що об’єкти в виділених контурах представляються як топологічна поверхня. Яскравість пікселів пропорційна висоті віртуального ландшафту об’єкту. Потім алгоритм починає заливати ці об’єкти “віртуальною” водою. В результаті, коли вода з декількох басейнів (з ділянок різних об’єктів) починає зливатись і досягає найвищих точок, утворюється лінії вододілу, які і є границями сегментованих об'єктів.

Також додатково було вивчено один із можливих способів покращення детекції об’єктів, використовуючи для цього модель семантичної сегментації. Модель семантичної сегментації - це нейромережа, що вміє виділяти області зображень за певними ознаками. В нашому випадку, на фото з Bing Map, цими ознаками могли бути наявність стін (яскравої, білої лінії), певна область даху та наявність тіні (темної, чорної лінії). Одна з найпопулярніших рішень є інструмент **U-Net**.

**Результати виконання лабораторної роботи:**
В ході виконання лабораторної роботи, я реалізував середовище, для детекції будівель на супутникових / аеро зображеннях. Програма дозволяє власноруч налаштувати параметри заданого пайплайну, для детекції будівель, за допомогою спеціальних повзунків та переключатись між результатом детекції на оригінальному фото та згенерованою маскою.

Фактично, реалізований пайплайн відповідає умовам виконання лабораторної роботи №3 (Доповнити результати лр2 методами фільтрації та корекції кольору цифрових зображень та розробити / модифікувати скрипт з функціоналом.)

**Структура проекту:**

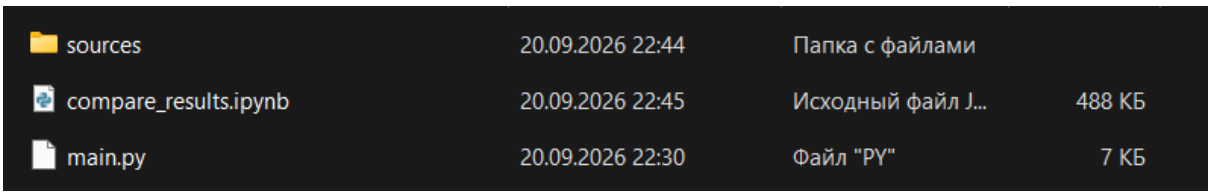



Рис. 1. Структура проекту для виконання лабораторної роботи №3

- **sources** - директорія з супутниковими зображеннями для детекції, та еталоном детекції, для порівняння.

- **compare_results.ipynb** - файл Jupyter Notebook для зручного аналізу результатів програмної детекції будівель
- **main.py** - виконавчий файл, що запускає виконання пайплайну, разом з інтерфейсом для корекції параметрів.

**Скріншоти роботи програми:**

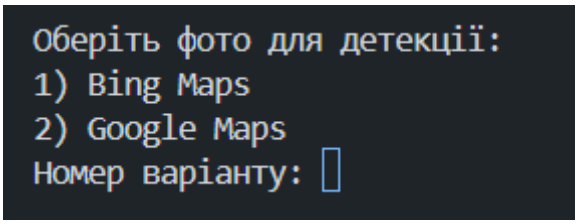



Рис. 2. Вибір сценарію (над яким фото будуть виконуватись операції для детекції будівель)

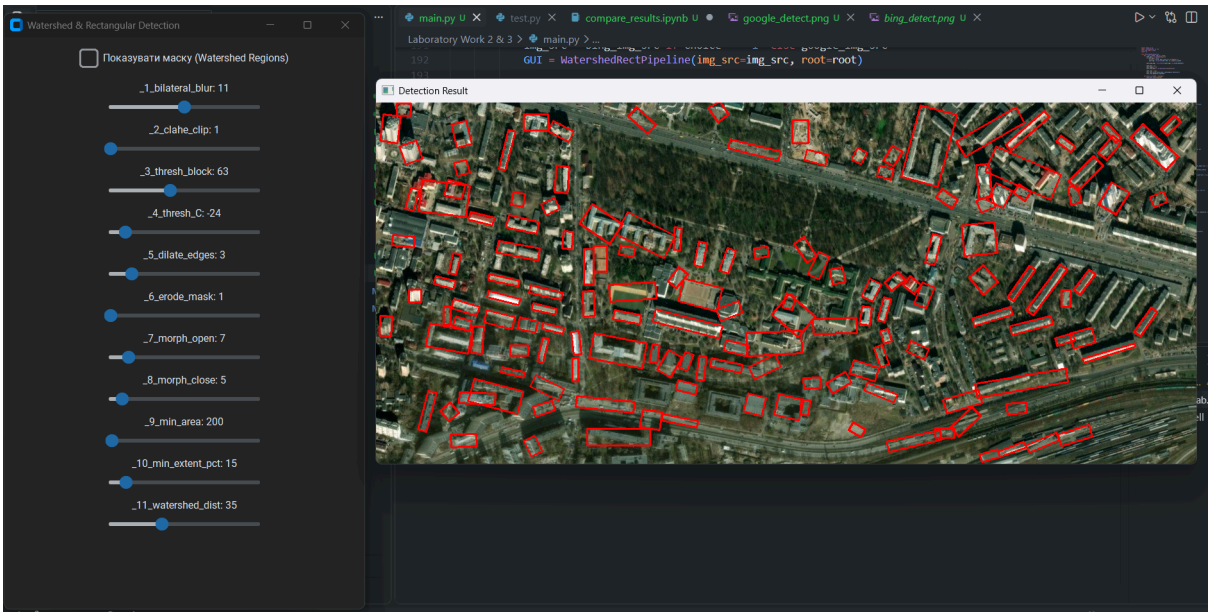



Рис. 3. Інтерфейс вікна налаштувань параметрів детекції та самих результатів детекції будівель на супутниковому знімку.

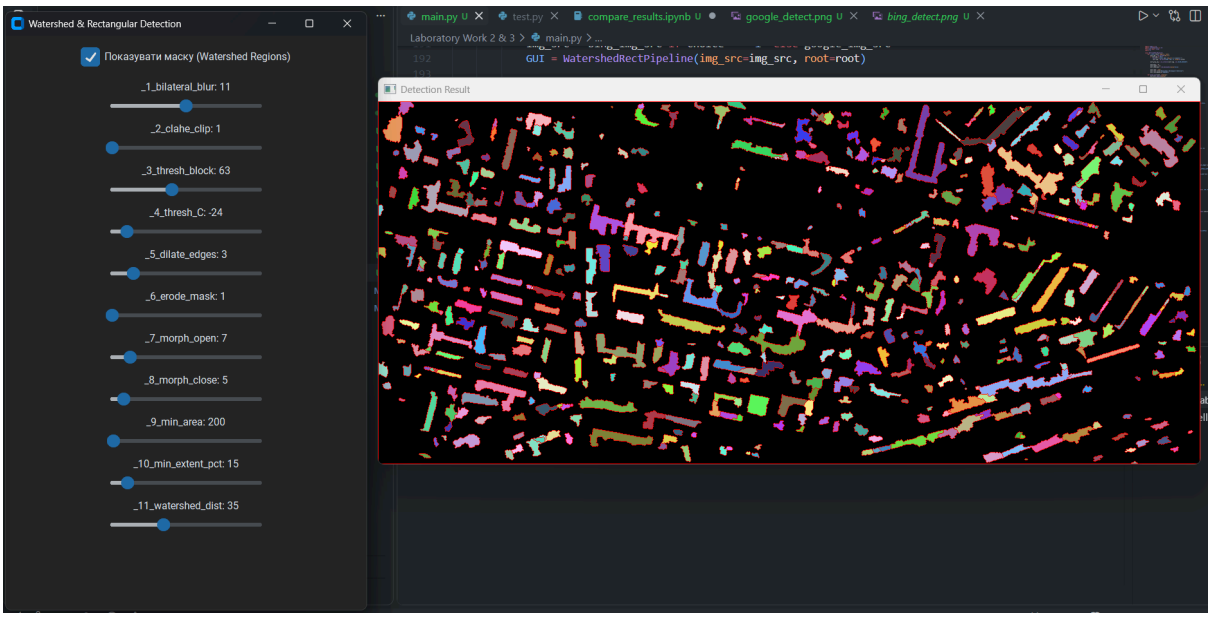



Рис. 4. Режим перегляду маски, для налагодження параметрів детекції будівель на супутниковому знімку.

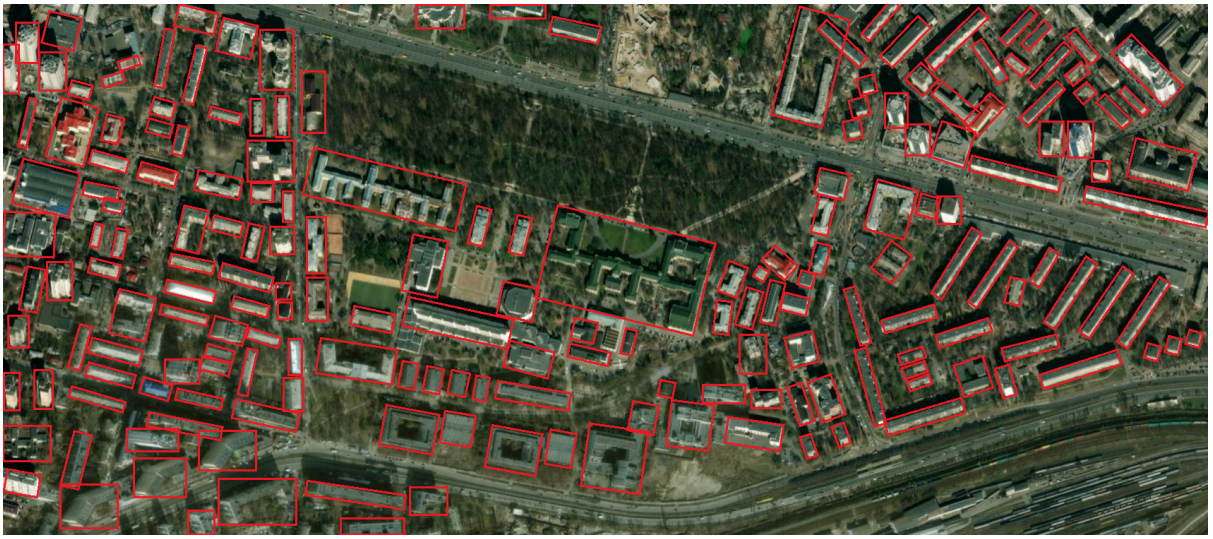


Рис. 5. Еталон роботи програми, побудований вручну для аналізу точності програмної детекції будівель.

Результати аналізу точності роботи програмної детекції будівель, порівнюючи області еталону та результату виконання програми:

**Пояснення параметрів аналізу:**

- **Total GT** - загальна кількість будівель, знайдена на еталоні
- **Total Preds** - загальна кількість будівель, знайдена програмно
- **TP (Знайдено)** - кількість будівель, що була знайдена програмно і співпала з поміченими областями на еталоні
- **FP (Хибні)** - кількість будівель, що була знайдена програмно проте не співпадає з поміченими областями на еталоні

- **FN (Пропуски)** - кількість будівель, що не була знайдена програмно проте є поміченими на еталоні
- **Фрагментовані (Over-segmented)** - кількість фрагментованих будівель (замість однієї суцільної області будівлі - область з декількох будівель)
- **Precision** - параметр точності детекції (відсоток того, що знайдені програмно області дійсно є шуканими об’єктами)
- **Recall** - параметр повноти детекції (відсоток, який характеризує загальну кількість знайдених реальних об’єктів загалом)
- **F1-Score** - параметр загальної оцінки детекції будівель

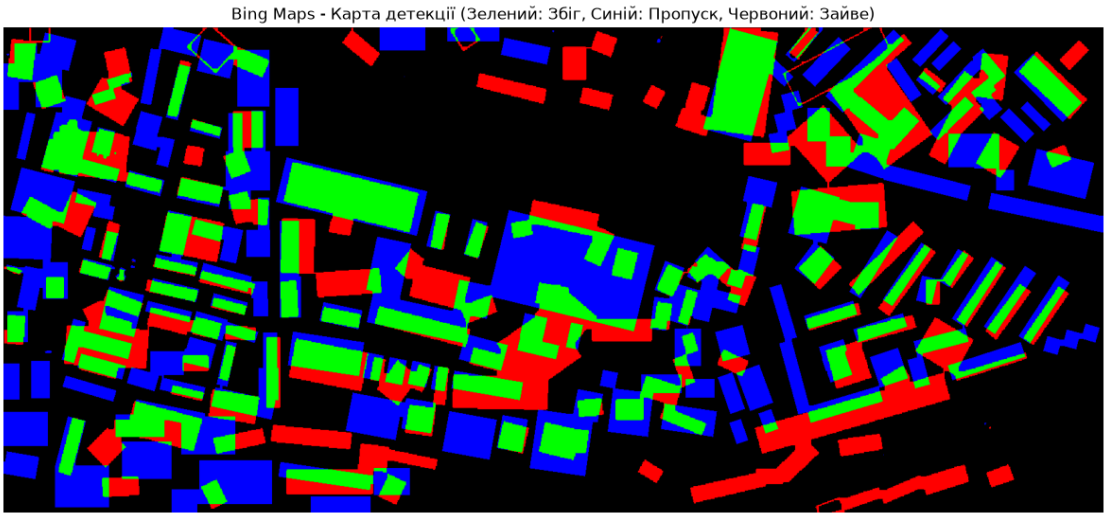



Рис. 6. Мапа порівняння детекції будівель на супутниковому зображенні на Bing Maps та еталону

==========

РЕЗУЛЬТАТИ АНАЛІЗУ: Bing Maps

==========

Авто-вирівнювання: зсув на X=0.18px, Y=0.03px
Total GT: 114
Total Preds: 90
TP (Знайдено): 62
FP (Хибні): 33
FN (Пропуски): 52
Фрагментовані (Over-segmented): 16
**Precision: 65.26%**
**Recall: 54.39%**
**F1-Score: 59.33%**

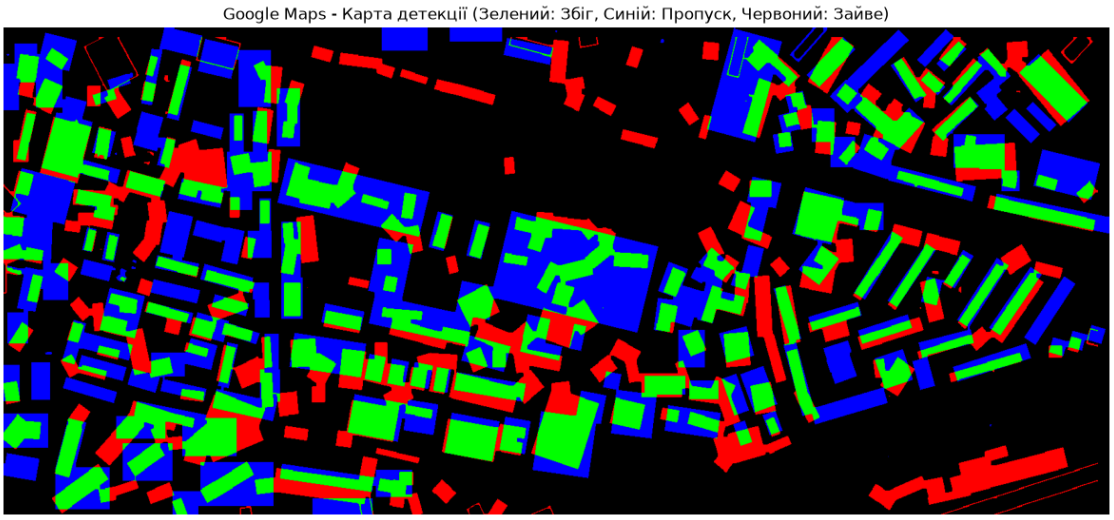



Рис. 7. Мапа порівняння детекції будівель на супутниковому зображенні на Google Maps та еталону

==========

РЕЗУЛЬТАТИ АНАЛІЗУ: Google Maps

==========

Авто-вирівнювання: зсув на X=-16.88px, Y=14.99px

Total GT: 114

Total Preds: 224

TP (Знайдено): 70

FP (Хибні): 141

FN (Пропуски): 44

Фрагментовані (Over-segmented): 25

**Precision: 33.18%**

**Recall: 61.4%**

**F1-Score: 43.08%**

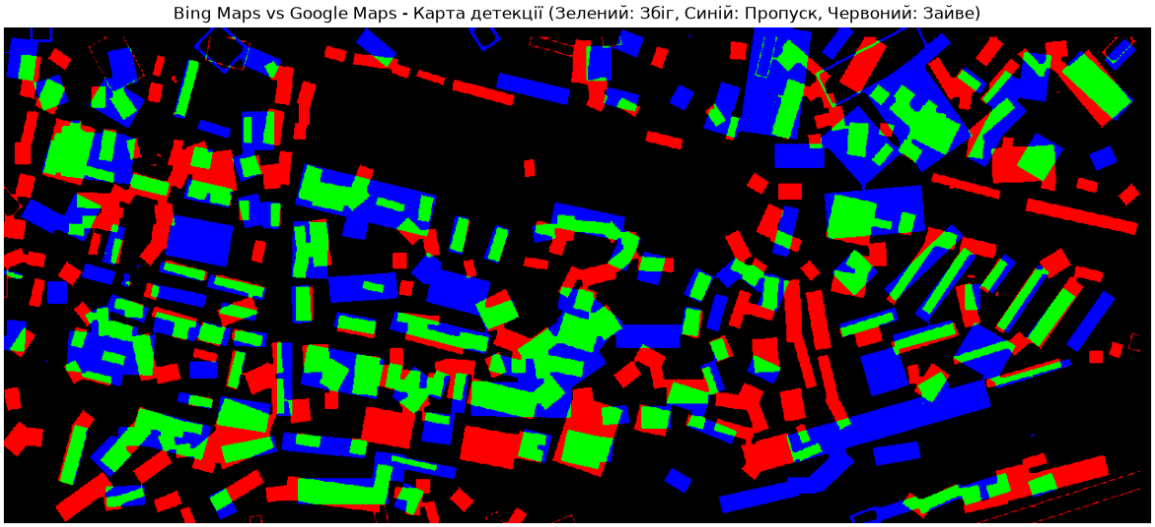



Рис. 8. Мапа збіжності результатів детекції між фото Bing Maps та Google Maps (за еталон взято Bing Maps)

==========
 РЕЗУЛЬТАТИ АНАЛІЗУ: Bing Maps vs Google Maps
==========
Авто-вирівнювання: зсув на X=-11.95px, Y=10.36px
Total GT: 91
Total Preds: 202
TP (Знайдено): 55
FP (Хибні): 123
FN (Пропуски): 36
Фрагментовані (Over-segmented): 16
**Precision: 30.9%**
**Recall: 60.44%**
**F1-Score: 40.89%**

В ході аналізу ми бачимо, що програмна реалізація, загального пайплайну детекції будівель для фото з Bing Maps відпрацювала загалом краще. Це відбулось через те, що фотографія з Bing Maps, зроблена під кутом. Тобто фактично ми отримуємо додаткову ознаку для будівлі, а саме білу лінії стінки.

У випадку з Google Maps, фото зроблено у форматі ортофотопланого зображення, тобто з усунутим спотворенням від нахилу камери. Через це, частина будівель може зливатись з фоном доріг, площ, тощо, без додаткових орієнтирів для детекції.

**Код програмної реалізації:**
main.py

```
import cv2
import numpy as np
import customtkinter as ctk
from time import sleep
import os

class WatershedRectPipeline:
    '''Загальний клас для інтерфейсу та пайплайну виконання детекції будівель'''
    def __init__(self, img_src, root):
        self.img = cv2.imread(img_src)
        if self.img is None:
            raise FileNotFoundError(f"Фото за шляхом {img_src} не знайдено!")
```

```
        self.gray_img = cv2.cvtColor(self.img, cv2.COLOR_BGR2GRAY) #
Зчитування фото у форматі відтінків сірого

        self.attr = {} # Словник аттрибутів параметрів для пайплайну
        self.labels = {} # Словник для написів над повзунками зміни
параметрів
        self.show_mask = ctk.BooleanVar(value=False) # Флаг для
переключення режиму перегляду між маскою та результатами детекції

        # Блок налаштування інтерфейсу
        self.root = root
        self.root.title("Watershed & Rectangular Detection")
        self.root.geometry("450x950")

    def param_init(self, **kwargs):
        '''Метод динамічної ініціалізації параметрів'''
        self.attr.update(kwargs)

    def interface_init(self):
        '''Метод ініціалізації інтерфейсу налаштувань та перегляду
результатів'''
        checkbox = ctk.CTkCheckBox(
            master=self.root,
            text="Показувати маску (Watershed Regions)",
            variable=self.show_mask,
            command=self.pipeline
        )
        checkbox.pack(pady=(15, 10))

        for key, values in self.attr.items():
            minimum, value, maximum, step_val = values
            label = ctk.CTkLabel(master=self.root, text=f"{key}:
{value}")
            label.pack(pady=(2, 0))
            self.labels[key] = label

            # Динамічне створення слайдерів для зміни відповідних
параметрів
            slider = ctk.CTkSlider(
                master=self.root,
                from_=minimum,
                to=maximum,
```

```
                number_of_steps=int((maximum - minimum) / step_val) if
step_val > 0 else None,
                command=lambda val, k=key: self.on_slider_change(k,
val)
            )
            slider.pack(pady=(2, 5))
            slider.set(value)

    def on_slider_change(self, key, value):
        '''Метод обробки при зміні значення на слайдері'''
        valid_value = self.value_check(key, value)
        self.attr[key][1] = valid_value
        self.labels[key].configure(text=f"{key}: {valid_value}")
        self.pipeline()

    def value_check(self, key, raw_value):
        '''Метод перевірки корректності введених через слайдер даних'''
        minimum, _, maximum, step = self.attr[key]
        if step == 0: return raw_value
        steps_count = round((raw_value - minimum) / step)
        snapped_value = minimum + (steps_count * step)
        snapped_value = max(minimum, min(snapped_value, maximum))
        return int(snapped_value) if isinstance(step, int) else
snapped_value

    def pipeline(self):
        '''Основний метод для виконання загального алгоритму детекції
будівель'''
        # Отримання базових параметрів
        blur_k = self.attr['_1_bilateral_blur'][1]
        thresh_block = self.attr['_3_thresh_block'][1]
        thresh_C = self.attr['_4_thresh_C'][1]
        open_k = self.attr['_7_morph_open'][1]
        close_k = self.attr['_8_morph_close'][1]
        min_area = self.attr['_9_min_area'][1]

        # Extent (Прямокутність)
        min_extent = self.attr['_10_min_extent_pct'][1] / 100.0
        watershed_dist = self.attr['_11_watershed_dist'][1] / 100.0

        # Підготовка та поріг
```

```
        blurred = cv2.bilateralFilter(self.gray_img, blur_k, 50, 50) # Використання Білатерального фільтру для усунення зайвих шумів зі збереженням контрасту границь об'єктів
        clahe = cv2.createCLAHE(clipLimit=self.attr['_2_clahe_clip'][1], tileGridSize=(8, 8)) # Коррекція контрасту границь об'єктів
        contrasted = clahe.apply(blurred)

        thresh_block_size = max(3, thresh_block | 1) # Ядор адаптивної бінарізації
        thresh = cv2.adaptiveThreshold( # Блок адаптивної бінарізації
            contrasted, 255,
            cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
            cv2.THRESH_BINARY,
            thresh_block_size, thresh_C
        )

        # Морфологія та монолітна маска
        edges = cv2.Canny(thresh, 50, 150) # Детекція границь Кенні
        k_dilate = cv2.getStructuringElement(cv2.MORPH_RECT, (max(1, self.attr['_5_dilate_edges'][1] | 1),)*2)
        dilated_edges = cv2.dilate(edges, k_dilate, iterations=1)

        # Блок малювання знайдних границь об'єктів
        raw_contours, _ = cv2.findContours(dilated_edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        solid_mask = np.zeros_like(self.gray_img)
        cv2.drawContours(solid_mask, raw_contours, -1, 255, -1)

        # Блок морфологічних операцій
        k_erode = cv2.getStructuringElement(cv2.MORPH_RECT, (max(1, self.attr['_6_erode_mask'][1] | 1),)*2) # Операція Ерозії
        separated_mask = cv2.erode(solid_mask, k_erode, iterations=1)

        # Ядра для морфологічних операцій
        k_open = cv2.getStructuringElement(cv2.MORPH_RECT, (max(1, open_k | 1),)*2)
        k_close = cv2.getStructuringElement(cv2.MORPH_RECT, (max(1, close_k | 1),)*2)

        cleaned_mask = cv2.morphologyEx(separated_mask, cv2.MORPH_OPEN, k_open) # Морфологічна операція відкриття
```

```
        final_solid_mask = cv2.morphologyEx(cleaned_mask,
cv2.MORPH_CLOSE, k_close) # Морфологічна операція закриття

        # Сегментація методом вододілу
        sure_bg = cv2.dilate(final_solid_mask, np.ones((3,3),
np.uint8), iterations=3)
        dist_transform = cv2.distanceTransform(final_solid_mask,
cv2.DIST_L2, 5)

        ret, sure_fg = cv2.threshold(dist_transform, watershed_dist *
dist_transform.max(), 255, 0)
        sure_fg = np.uint8(sure_fg)

        unknown = cv2.subtract(sure_bg, sure_fg)

        ret, markers = cv2.connectedComponents(sure_fg)
        markers = markers + 1
        markers[unknown == 255] = 0

        blurred_bgr = cv2.bilateralFilter(self.img, blur_k, 50, 50)
        markers = cv2.watershed(blurred_bgr, markers)

        if self.show_mask.get():
            display_mask = np.zeros_like(self.img)
            display_mask[markers == -1] = [0, 0, 255]

            for i in range(2, ret + 1):
                color = (np.random.randint(50, 255),
np.random.randint(50, 255), np.random.randint(50, 255))
                display_mask[markers == i] = color

            cv2.imshow("Detection Result", display_mask)
            cv2.waitKey(1)
            return

        # Векторизація прямокутниками (minAreaRect)
        output_img = self.img.copy()
        valid_buildings_count = 0

        for marker_id in range(2, ret + 1):
            obj_mask = np.zeros_like(final_solid_mask)
            obj_mask[markers == marker_id] = 255
```

```
            contours, _ = cv2.findContours(obj_mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

            if not contours:
                continue

            cnt = contours[0]
            area = cv2.contourArea(cnt)

            if area > min_area:
                # Отримуємо мінімальний прямокутник
                rect = cv2.minAreaRect(cnt)
                width, height = rect[1]

                if width == 0 or height == 0:
                    continue

                rect_area = width * height
                extent = area / rect_area

                # Відсіюємо сміття за параметром Extent
                if extent >= min_extent:
                    valid_buildings_count += 1
                    # Отримуємо 4 кути прямокутника і малюємо його
                    box = cv2.boxPoints(rect)
                    box = np.int32(box)
                    cv2.drawContours(output_img, [box], 0, (0, 0, 255),
2)

        os.system('cls' if os.name == 'nt' else 'clear')
        print(f"Знайдено будівель (Watershed + Rectangles):
{valid_buildings_count}")

        cv2.imshow("Detection Result", output_img)
        cv2.waitKey(1)

if __name__ == "__main__":
    bing_img_src = r"./sources/bing_maps.png"
    google_img_src = r"./sources/google_maps.png"

    ctk.set_appearance_mode("Dark")
    root = ctk.CTk()
```

```
    while True:
        os.system('cls' if os.name == 'nt' else 'clear')
        print("Оберіть фото для детекції:\n1) Bing Maps\n2) Google
Maps")
        choice = input("Номер варіанту: ")

        if choice in ['1', '2']:
            img_src = bing_img_src if choice == '1' else google_img_src
            GUI = WatershedRectPipeline(img_src=img_src, root=root) #
Ініціаліація екземпляру класу інтерфейсу

            GUI.param_init( # Ініціалізація необхіднхи параметрів для
пайплайну
                _1_bilateral_blur=[1, 11, 21, 2],
                _2_clahe_clip=[1, 1, 10, 1],
                _3_thresh_block=[3, 63, 151, 2],
                _4_thresh_C=[-30, -24, 30, 1],
                _5_dilate_edges=[1, 3, 15, 2],
                _6_erode_mask=[1, 1, 21, 2],
                _7_morph_open=[1, 7, 51, 2],
                _8_morph_close=[1, 5, 51, 2],
                _9_min_area=[100, 200, 10000, 50],
                _10_min_extent_pct=[5, 15, 100, 5],
                _11_watershed_dist=[1, 35, 99, 1]
            )
            break
        else:
            print("Спробуйте ще раз!")
            sleep(1)

    GUI.interface_init() # Ініціалізація інтерфейсу
    GUI.pipeline() # Запуск загального алгоритму детекції будівель
    root.mainloop()
```

**compare_results.ipynb**

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def extract_filled_boxes(image_path):
    """
    Завантажує зображення, знаходить червоні рамки та перетворює їх в
сплошну бінарну маску.
```

```
    """
    img = cv2.imread(image_path)
    if img is None:
        raise FileNotFoundError(f"Не знайдено файл: {image_path}")

    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

    # Червоний колір в HSV знаходится на двух краях спектру (0-10 та
170-180)
    lower_red1 = np.array([0, 70, 50])
    upper_red1 = np.array([10, 255, 255])
    lower_red2 = np.array([170, 70, 50])
    upper_red2 = np.array([180, 255, 255])

    mask1 = cv2.inRange(hsv, lower_red1, upper_red1)
    mask2 = cv2.inRange(hsv, lower_red2, upper_red2)
    red_mask = mask1 | mask2

    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
    red_mask = cv2.morphologyEx(red_mask, cv2.MORPH_CLOSE, kernel,
iterations=2)

    # Знаходимо контури рамок і заливаємо їх всередині
    contours, _ = cv2.findContours(red_mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    filled_mask = np.zeros_like(red_mask)
    cv2.drawContours(filled_mask, contours, -1, 255, -1)

    return img, filled_mask

def evaluate_detection(gt_mask, pred_mask, overlap_threshold=0.4):
    """
    Порівнює маску еталону та маску детекції з урахуванням проблеми
Over-segmentation.
    """
    # Знаходимо окремі будівлі на еталоні (Ground Truth)
    num_gt, gt_labels, gt_stats, _ =
cv2.connectedComponentsWithStats(gt_mask, connectivity=8)
    # Знаходимо окремі будівлі на програмній детекції (Predictions)
    num_pred, pred_labels, pred_stats, _ =
cv2.connectedComponentsWithStats(pred_mask, connectivity=8)

    tp = 0
```

```
    over_segmented = 0
    matched_preds = set()

    # Перевіряємо кожну будівлю з еталону (індекс 0 - це фон, тому
пропускаємо)
    for i in range(1, num_gt):
        gt_area = gt_stats[i, cv2.CC_STAT_AREA]

        # Виділямєо тільки текущее эталонное здание
        single_gt_mask = (gt_labels == i).astype(np.uint8)

        # Знаходимо перетин будівлі з програмною маскою
        intersection = cv2.bitwise_and(single_gt_mask, pred_mask)
        intersect_area = np.count_nonzero(intersection)

        # Якщо прогрмні рамки покривають хоча б 40% еталоної будівлі
        if (intersect_area / gt_area) >= overlap_threshold:
            tp += 1

            intersecting_preds = np.unique(pred_labels[intersection >
0])
            intersecting_preds = intersecting_preds[intersecting_preds
> 0] # Прибираємо фон

            matched_preds.update(intersecting_preds)

            # Якщо будівля зацепила 2 и більше рамок — це
over-segmentation
            if len(intersecting_preds) > 1:
                over_segmented += 1

    # FN (False Negatives) - будівлі на еталоні, які не були знайдені
програмно
    fn = (num_gt - 1) - tp

    # FP (False Positives) - програмні рамки, які ні з чим не
перетинаются
    fp = (num_pred - 1) - len(matched_preds)

    # Обрахунок значень
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0
```

```
    f1_score = 2 * (precision * recall) / (precision + recall) if
(precision + recall) > 0 else 0

    return {
        "Total GT": num_gt - 1,
        "Total Preds": num_pred - 1,
        "TP (Знайдено)": tp,
        "FP (Хибні)": fp,
        "FN (Пропуски)": fn,
        "Фрагментовані (Over-segmented)": over_segmented,
        "Precision": round(precision * 100, 2),
        "Recall": round(recall * 100, 2),
        "F1-Score": round(f1_score * 100, 2)
    }

def analyze_and_visualize(etalon_path, pred_path, title):
    # Витягуємо маски та зображення
    gt_img, gt_mask = extract_filled_boxes(etalon_path)
    pred_img, pred_mask = extract_filled_boxes(pred_path)

    # Блок виправлення розмірів
    height, width = gt_mask.shape
    if pred_mask.shape != gt_mask.shape:
        pred_img = cv2.resize(pred_img, (width, height),
interpolation=cv2.INTER_LINEAR)
        pred_mask = cv2.resize(pred_mask, (width, height),
interpolation=cv2.INTER_NEAREST)

    # Блок авто-вирівнювання:
    # Переводимо в градації сірого для пошуку зсуву
    gt_gray = cv2.cvtColor(gt_img, cv2.COLOR_BGR2GRAY)
    pred_gray = cv2.cvtColor(pred_img, cv2.COLOR_BGR2GRAY)

    # Фазова кореляція знаходить точний зсув (x, y) між двома
картинками у пікселях
    shift, _ = cv2.phaseCorrelate(np.float64(pred_gray),
np.float64(gt_gray))
    dx, dy = shift

    # Створюємо матрицю трансформації та зсуваємо маску детекції на
знайдені пікселі
    M = np.float32([[1, 0, dx], [0, 1, dy]])
    aligned_pred_mask = cv2.warpAffine(pred_mask, M, (width, height))
```

```
    # Оцінюємо вже ідеально вирівняні маски
    results = evaluate_detection(gt_mask, aligned_pred_mask)

    # Вивід результатів
    print(f"\n{'='*40}")
    print(f" РЕЗУЛЬТАТИ АНАЛІЗУ: {title}")
    print(f"{'='*40}")
    print(f"Авто-вирівнювання: зсув на X={round(dx, 2)}px, Y={round(dy,
2)}px")
    for key, value in results.items():
        if "Precision" in key or "Recall" in key or "F1" in key:
            print(f"{key}: {value}%")
        else:
            print(f"{key}: {value}")

    # Візуалізація різниці
    diff_img = np.zeros_like(gt_img)
    diff_img[gt_mask == 255] = [255, 0, 0]     # Синій - Пропуски
    diff_img[aligned_pred_mask == 255] = [0, 0, 255]  # Червоний -
Хибні (мусор)
    diff_img[(gt_mask == 255) & (aligned_pred_mask == 255)] = [0, 255,
0] # Зелений - Збіг

    diff_img_rgb = cv2.cvtColor(diff_img, cv2.COLOR_BGR2RGB)

    plt.figure(figsize=(15, 7))
    plt.imshow(diff_img_rgb)
    plt.title(f"{title} - Карта детекції (Зелений: Збіг, Синій:
Пропуск, Червоний: Зайве)")
    plt.axis('off')
    plt.show()

# Запуск аналізу
etalon_file = r"./sources/etalon_detection.png"

# Аналіз Bing Maps
bing_result_file = r"./sources/bing_detect.png"
analyze_and_visualize(etalon_file, bing_result_file, "Bing Maps")

# Аналіз Google Maps
google_result_file = r"./sources/google_detect.png"
analyze_and_visualize(etalon_file, google_result_file, "Google Maps")
```

```
# Аналіз схожості результатів Bing Maps та Google Maps
analyze_and_visualize(bing_result_file, google_result_file, "Bing Maps vs Google Maps")
```

**Висновок**: На цій лабораторній роботі, ми розібрались з методами обробки графічних зображень та синтезували власний загальний алгоритм обробки супутникового зображення, для детекції будівель на фото. В ході виконання роботи було вивчено різноманітні моделі представлення зображень (RGB, BGR, HSV, GRAYSCALE, монохром), методи фільтрації зображення, їх сильні та слабкі сторони, методи бінарізації та векторизації зображення. Також розібрали математичне обгрунтування відповідних алгоритмів, проаналізували причини виникнення похибок результатів виконання нашого алгоритму та оглядово розібрали актуальні комерційні рішення для покращення результатів детекції об’єктів при ДЗЗ.

В результаті виконання даної роботи, ми отримали робочий алгоритм для детекції будівель, який з ~60% ефективністю знаходить будівлі, для супутникових / аеро зображень під кутом. Також алгоритм фактично є універсальним. Він може обробляти як і знімки під кутом, так і ортофотопланові знімки, хоч і з частковою втратою ефективності детекції, а реалізований клас інтерфейсу з методом під загальний алгоритм детекції, робить синтезований код гнучким для модифікації і зручним для експериментів для підбору параметрів, в залежності від алгоритму та параметрів наданої фотографії.